Practical Challenge: Multi-Source Chatbot with Intelligent Agent Orchestration

Marcelo Fungueiro (CraftLabs)
SR Operation Manager

Intro

Your mission is to design and build a chatbot capable of answering questions **and** performing actions — intelligently deciding when to retrieve information and when to invoke external tools.

You may choose any use case: an internal assistant, a project manager helper, a contractor assignment bot, or anything else.

Informational Queries (RAG + Knowledge Graph + Centralized Agent Flow)

For user questions that seek information, your agent must automatically gather and synthesize data from multiple sources:

- **Documents or text corpora** → via **RAG**
- **Structured facts and entity relationships** → via **Knowledge Graph (KG)**

These informational queries must always flow through a **centralized graph/agent pipeline**, capable of combining RAG and KG results.

Valid orchestration options (choose any):

- **LangGraph**
- **LangChain (ReAct-style agents, tool-calling agents, custom graphs, etc.)**
- **CrewAI**

- Any equivalent framework capable of coordinating multi-step reasoning and tool use

The agent must decide autonomously when a query is informational.

No keyword-based routing or if/else logic is allowed.

Action Requests (Mandatory MCP Server Integration)

For queries where the user requests the system to perform an action — e.g., updating a record, assigning something, triggering a workflow — the agent **must** call an action tool implemented through an **MCP Server**.

Requirements:

- You must implement an **MCP Server** exposing one or more tools
- The agent must automatically decide when to call the MCP tool
- The MCP Server must not be part of the informational graph (RAG/KG)
- Actions and informational queries must remain strictly separated
- The agent should adapt to new MCP tools without modifying decision logic

The MCP Server is the **only valid mechanism** for executing actions.

User Interface

You may use any existing chat UI, as long as it connects to your agent without requiring custom frontend development.

Allowed options include:

- **LangChain Agent Chat UI** (recommended)
- **LangGraph Chat UI**
- **Gradio**
- Terminal chat interfaces
- Any ready-made chat UI that forwards messages and prints responses

The UI must remain passive:

the agent decides whether to invoke the informational graph or the MCP Server.

Optional (inside this point) — MCP Server as external integration

As an extension, you may optionally expose your solution as an MCP Server that can be consumed by clients such as:

- GitHub Copilot
- VS Code MCP

- Other MCP-enabled interfaces

This is optional and not required unless you want broader integration.

Evaluation Criteria

Your solution is considered valid if:

- Informational queries go through a centralized RAG + KG agent flow
- Action requests are always executed through an MCP Server
- The agent selects between “informational” vs. “action” routes autonomously
- No manual conditional logic (if/else, keyword matching, etc.)
- Architecture is modular, extensible, and tools can be added without modifying routing logic
- The MCP Server handles all actions and uses correct MCP schemas

Useful links

- RAG - https://www.youtube.com/watch?v=uAsd9pOIcLg
- Knowledge Graph - https://www.youtube.com/watch?v=O-T_6KOXML4